

Задание 1.1. Cursor IDE

Создание проекта.

Cursor - редактор кода на базе VS Code со встроенным AI-агентом. Установка выполняется с официального сайта cursor.com: скачивается установщик под текущую ОС (Windows/macOS/Linux) и запускается стандартным способом. Системные требования минимальны: 4 ГБ ОЗУ, 2 ГБ свободного места. После запуска предлагается авторизация через email, Google или GitHub. При переходе с VS Code можно импортировать настройки и плагины опцией «Import from VS Code». Новый проект создаётся через File > Open Folder - выбирается папка с кодом, Cursor индексирует структуру и делает её доступной AI-агенту.

Кодинг. Основной режим работы - Agent Mode: достаточно описать задачу текстом, и AI самостоятельно анализирует код, создаёт файлы, пишет код и запускает команды. Для точечных правок используется инлайн-режим Cmd+K (macOS) или Ctrl+K (Windows) выделяется фрагмент, вводится инструкция, AI применяет изменение на месте. Ссылки @ дают точный контекст: @file.ts - конкретный файл, @functionName - функция, @docs - документация. При написании кода AI предсказывает следующие строки, принимаемые по Tab.

Форматирование. Cursor наследует настройки форматирования VS Code. В settings.json включается editor.formatOnSave: true для автоформата при сохранении. Стил настраивается в Settings | Editor | Code Style. При наличии файла.editorconfig в проекте IDE автоматически применяет указанные в нём правила.

Отладка. Для диагностики ошибок используется чат с контекстом терминала: копируется stack trace, добавляется ссылка @terminal, задаётся вопрос «почему падает и как исправить» Cursor сопоставляет ошибку с кодом проекта и предлагает конкретное решение. Инлайн-команды // debug: why is this throwing NullPointerException? и // lint: fix ESLint warnings позволяют исправлять

ошибки и стилевые замечания без переключения контекста. Стандартные точки останова и панель переменных работают как в VS Code.

Запуск и компиляция. Cursor автоматически выполняет терминальные команды, но по умолчанию запрашивает подтверждение. Для C/C++ создаётся `tasks.json` с командой компиляции (`g++ -std=c++17 -g ${file} -o ${fileDirname}/${fileBasenameNoExtension}`) и `launch.json` для отладки через GDB. Компиляция запускается по `Ctrl+Shift+B`, запуск - через панель Run or кнопку в терминале. Для других языков используются штатные конфигурации VS Code.